

```

import os
import sys
import time
import requests
import logging
import threading
from datetime import datetime
from datetime import timezone
from flask import Flask
from flask import jsonify

log_format = "%(asctime)s %(message)s"
logging.basicConfig(level=logging.INFO, format=log_format, stream=sys.stdout, for
log = logging.getLogger(__name__)

TOKEN_KEY = "TELEGRAM_BOT_TOKEN"
CHAT_KEY = "TELEGRAM_CHAT_ID"
LIQ_KEY = "MIN_LIQUIDITY"
VOL_KEY = "MIN_VOLUME_24H"
AGE_KEY = "MAX_TOKEN_AGE_H"
INT_KEY = "SCAN_INTERVAL"
SCORE_KEY = "MIN_SIGNAL_SCORE"

TELEGRAM_BOT_TOKEN = os.environ.get(TOKEN_KEY, "")
TELEGRAM_CHAT_ID = os.environ.get(CHAT_KEY, "")
MIN_LIQUIDITY = float(os.environ.get(LIQ_KEY, 10000))
MIN_VOLUME_24H = float(os.environ.get(VOL_KEY, 5000))
MAX_TOKEN_AGE_H = float(os.environ.get(AGE_KEY, 24))
SCAN_INTERVAL = float(os.environ.get(INT_KEY, 30))
MIN_SIGNAL_SCORE = float(os.environ.get(SCORE_KEY, 65))

app = Flask(__name__)
sent_tokens = set()
signal_count = 0
last_scan = None

def send_telegram(message):
    if not TELEGRAM_BOT_TOKEN or not TELEGRAM_CHAT_ID:
        log.warning("Token ou Chat ID manquant")
        return False
    tg_url = "https://api.telegram.org/bot" + TELEGRAM_BOT_TOKEN + "/sendMessage"
    payload = dict()
    payload["chat_id"] = TELEGRAM_CHAT_ID
    payload["text"] = message

```

```

payload["parse_mode"] = "HTML"
payload["disable_web_page_preview"] = True
try:
    r = requests.post(tg_url, json=payload, timeout=10)
    if r.status_code == 200:
        log.info("Telegram OK")
        return True
    log.error("Telegram erreur %s", r.status_code)
except Exception as e:
    log.error("Telegram exception: %s", e)
return False

def check_rug_solana(address):
    try:
        api = "https://api.rugcheck.xyz/v1/tokens/" + address + "/report/summary"
        r = requests.get(api, timeout=8)
        if r.status_code == 200:
            data = r.json()
            sc = data.get("score", 0)
            risks = data.get("risks", [])
            names = [x.get("name", "") for x in risks]
            if sc >= 80:
                return "SAFE", sc, names
            elif sc >= 50:
                return "WARN", sc, names
            else:
                return "DANGER", sc, names
        except Exception as e:
            log.error("Rugcheck erreur: %s", e)
        return "UNKNOWN", 0, []

def check_rug_evm(address):
    try:
        api = "https://api.tokensniffer.com/v2/tokens/" + address
        r = requests.get(api, timeout=8)
        if r.status_code == 200:
            data = r.json()
            sc = data.get("score", 0)
            if sc >= 70:
                return "SAFE", sc, []
            elif sc >= 40:
                return "WARN", sc, []
            else:
                return "DANGER", sc, []
        except Exception as e:
            log.error("TokenSniffer erreur: %s", e)
        return "UNKNOWN", 0, []

```

```

def get_rug_status(chain, address):
    if not address:
        return "UNKNOWN", 0, []
    if chain == "solana":
        return check_rug_solana(address)
    elif chain in ["bsc", "ethereum", "base"]:
        return check_rug_evm(address)
    return "UNKNOWN", 0, []

def score_token(pair):
    score = 0
    reasons = []
    liq = pair.get("liquidity", {}).get("usd", 0) or 0
    vol24 = pair.get("volume", {}).get("h24", 0) or 0
    vol1 = pair.get("volume", {}).get("h1", 0) or 0
    buys = pair.get("txns", {}).get("h24", {}).get("buys", 0) or 0
    sells = pair.get("txns", {}).get("h24", {}).get("sells", 0) or 0
    ch1 = pair.get("priceChange", {}).get("h1", 0) or 0
    ch6 = pair.get("priceChange", {}).get("h6", 0) or 0
    mc = pair.get("marketCap", 0) or 0
    vlr = vol24 / liq if liq > 0 else 0
    if vlr > 5:
        score += 25
        reasons.append("Volume/Liq x" + str(round(vlr, 1)) + " tres actif")
    elif vlr > 2:
        score += 15
        reasons.append("Volume/Liq x" + str(round(vlr, 1)))
    elif vlr > 1:
        score += 8
    total = buys + sells
    if total > 0:
        bp = buys / total
        if bp > 0.70:
            score += 20
            reasons.append(str(round(bp * 100)) + " pct achats bull fort")
        elif bp > 0.60:
            score += 12
            reasons.append(str(round(bp * 100)) + " pct achats")
        elif bp > 0.50:
            score += 5
    if ch1 > 20:
        score += 20
        reasons.append("+" + str(round(ch1, 1)) + " pct sur 1h")
    elif ch1 > 10:
        score += 12
        reasons.append("+" + str(round(ch1, 1)) + " pct sur 1h")

```

```

elif ch1 > 5:
    score += 6
elif ch1 < -10:
    score -= 10
if ch6 > 50:
    score += 15
    reasons.append("+" + str(round(ch6, 1)) + " pct sur 6h")
elif ch6 > 20:
    score += 8
avg = vol24 / 24 if vol24 > 0 else 0
if avg > 0 and vol1 > avg * 3:
    score += 15
    reasons.append("Vol 1h x" + str(round(vol1 / avg, 1)) + " vs moyenne")
elif avg > 0 and vol1 > avg * 1.5:
    score += 7
if 0 < mc < 100000:
    score += 10
    reasons.append("Micro-cap " + str(round(mc)))
elif 0 < mc < 500000:
    score += 6
    reasons.append("Small-cap " + str(round(mc)))
if 20000 < liq < 200000:
    score += 5
    reasons.append("Liquidite saine " + str(round(liq)))
return min(score, 100), reasons

def passes_filters(pair):
    liq = pair.get("liquidity", {}).get("usd", 0) or 0
    vol = pair.get("volume", {}).get("h24", 0) or 0
    crea = pair.get("pairCreatedAt")
    if liq < MIN_LIQUIDITY:
        return False, "Liq faible"
    if vol < MIN_VOLUME_24H:
        return False, "Vol faible"
    if crea:
        age_h = (time.time() * 1000 - crea) / 3600000
        if age_h > MAX_TOKEN_AGE_H:
            return False, "Trop vieux"
    return True, "OK"

def format_signal(pair, score, reasons, rug_status, rug_score, rug_risks):
    name = pair.get("baseToken", {}).get("name", "?")
    sym = pair.get("baseToken", {}).get("symbol", "?")
    addr = pair.get("baseToken", {}).get("address", "")
    chain = pair.get("chainId", "?").upper()
    dex = pair.get("dexId", "?").capitalize()
    price = pair.get("priceUsd", "0")

```

```

liq = pair.get("liquidity", {}).get("usd", 0) or 0
v24 = pair.get("volume", {}).get("h24", 0) or 0
v1 = pair.get("volume", {}).get("h1", 0) or 0
b24 = pair.get("txns", {}).get("h24", {}).get("buys", 0) or 0
s24 = pair.get("txns", {}).get("h24", {}).get("sells", 0) or 0
c1 = pair.get("priceChange", {}).get("h1", 0) or 0
c6 = pair.get("priceChange", {}).get("h6", 0) or 0
c24 = pair.get("priceChange", {}).get("h24", 0) or 0
mc = pair.get("marketCap", 0) or 0
dex_url = pair.get("url", "")
crea = pair.get("pairCreatedAt")
age = str(round((time.time() * 1000 - crea) / 3600000, 1)) + "h" if crea else
bar = "X" * int(score / 10) + "." * (10 - int(score / 10))
em = "FORT" if score >= 80 else "MOYEN"
chain_low = pair.get("chainId", "").lower()
if chain_low == "solana":
    rug_url = "https://rugcheck.xyz/tokens/" + addr
    buy_url = "https://jup.ag/swap/SOL-" + addr
elif chain_low == "bsc":
    rug_url = "https://tokensniffer.com/token/bsc/" + addr
    buy_url = "https://pancakeswap.finance/swap?outputCurrency=" + addr
elif chain_low == "base":
    rug_url = "https://tokensniffer.com/token/base/" + addr
    buy_url = "https://app.uniswap.org/swap?outputCurrency=" + addr
else:
    rug_url = "https://tokensniffer.com/token/eth/" + addr
    buy_url = "https://app.uniswap.org/swap?outputCurrency=" + addr
if rug_status == "SAFE":
    rug_line = "RUG CHECK: SAFE (score " + str(rug_score) + "/100)"
elif rug_status == "WARN":
    rug_line = "RUG CHECK: ATTENTION (score " + str(rug_score) + "/100)"
elif rug_status == "DANGER":
    rug_line = "RUG CHECK: DANGER (score " + str(rug_score) + "/100)"
else:
    rug_line = "RUG CHECK: Non disponible"
risks_line = ""
if rug_risks:
    risks_line = "Risques: " + ", ".join(rug_risks[:3]) + "\n"
rsn = "\n".join(" - " + r for r in reasons) or " - Criteres de base atteint"
sep = "-----"
lines = [
    em + " SIGNAL MEMECOIN",
    sep,
    "Nom: " + name + " ($" + sym + ")",
    "Chain: " + chain + " | DEX: " + dex + " | Age: " + age,
    "Prix: $" + str(price),
    "1h: " + str(round(c1, 1)) + "% | 6h: " + str(round(c6, 1)) + "% | 24h: "

```

```

        "Liquidite: $" + str(round(liq)),
        "Vol 24h: $" + str(round(v24)) + " | Vol 1h: $" + str(round(v1)),
        "MarketCap: $" + str(round(mc)),
        "Txns: " + str(b24) + " achats / " + str(s24) + " ventes",
        "Score: [" + bar + "]" + str(score) + "/100",
        rs_n,
        sep,
        rug_line,
        risks_line,
        sep,
        "Adresse (copie-coll ):",
        "<code>" + addr + "</code>",
        sep,
        "DexScreener: " + dex_url,
        "Rug Check: " + rug_url,
        "Acheter: " + buy_url,
        sep,
        "DYOR - Pas un conseil financier"
    ]
    return "\n".join(lines)

def fetch_new_pairs():
    all_pairs = []
    chains = ["solana", "bsc", "ethereum", "base"]
    for chain in chains:
        try:
            api = "https://api.dexscreener.com/latest/dex/tokens/trending/" + cha
            r = requests.get(api, timeout=10)
            if r.status_code == 200:
                pairs = r.json().get("pairs", []) or []
                all_pairs.extend(pairs)
                log.info("Chain %s: %d pairs", chain, len(pairs))
        except Exception as e:
            log.error("Erreur %s: %s", chain, e)
    if not all_pairs:
        try:
            r = requests.get("https://api.dexscreener.com/latest/dex/search?q=mem
            if r.status_code == 200:
                all_pairs = r.json().get("pairs", []) or []
        except Exception as e:
            log.error("Erreur fallback: %s", e)
    return all_pairs

def scan_loop():
    global signal_count, last_scan
    log.info("Bot démarre - scan toutes les %s secondes", SCAN_INTERVAL)
    msg_start = (

```

```

        "Bot MemeSignal démarre !\n\n"
        + "Liquidite min: $" + str(round(MIN_LIQUIDITY)) + "\n"
        + "Volume min: $" + str(round(MIN_VOLUME_24H)) + "\n"
        + "Age max: " + str(MAX_TOKEN_AGE_H) + "h\n"
        + "Score min: " + str(MIN_SIGNAL_SCORE) + "/100\n"
        + "Intervalle: " + str(SCAN_INTERVAL) + "s\n\n"
        + "Rug check automatique actif\n"
        + "Surveillance active..."
    )
    send_telegram(msg_start)
    while True:
        last_scan = datetime.now(timezone.utc).isoformat()
        try:
            pairs = fetch_new_pairs()
            log.info("Total: %d pairs analyses", len(pairs))
            for pair in pairs:
                addr = pair.get("pairAddress", "")
                if not addr or addr in sent_tokens:
                    continue
                ok, _ = passes_filters(pair)
                if not ok:
                    continue
                score, reasons = score_token(pair)
                if score < MIN_SIGNAL_SCORE:
                    continue
                token_addr = pair.get("baseToken", {}).get("address", "")
                chain = pair.get("chainId", "").lower()
                rug_status, rug_score, rug_risks = get_rug_status(chain, token_ad
                if rug_status == "DANGER":
                    log.info("Token ignore RUG DANGER: %s", pair.get("baseToken",
                    sent_tokens.add(addr)
                    continue
                msg = format_signal(pair, score, reasons, rug_status, rug_score,
                if send_telegram(msg):
                    sent_tokens.add(addr)
                    signal_count += 1
                    sym = pair.get("baseToken", {}).get("symbol", "?")
                    log.info("Signal #%d envoye: %s score=%d rug=%s", signal_coun
                    if len(sent_tokens) > 5000:
                        sent_tokens.clear()
            except Exception as e:
                log.error("Erreur scan: %s", e)
            time.sleep(SCAN_INTERVAL)

@app.route("/")
def index():
    return jsonify({"status": "running", "signals": signal_count, "last_scan": la

```

```
@app.route("/health")
def health():
    return jsonify({"ok": True})

log.info("Demarrage du thread bot...")
threading.Thread(target=scan_loop, daemon=True).start()

if __name__ == "__main__":
    port = int(os.environ.get("PORT", 8080))
    app.run(host="0.0.0.0", port=port)
```